

Exercices — La boucle « while » (2)**Exercice 1**

Le script ci-dessous détermine le plus petit entier naturel n tel que la somme des carrés de 0 à n soit supérieure ou égale à 1 000.

```
1 n = 1
2 s = 0
3 while s < 1000:
4     s = s + n**2
5     n = n + 1
6 print(n)
```

1. Expliquer l'utilité de la variable n .
2. Que se passe-t-il si on inverse les deux instructions de la boucle ?

Exercice 2

Un jardinier souhaite creuser un puits. Le premier mètre coûte 100 €, le second 20 €, le troisième 40 € et ainsi de suite en ajoutant 20 € tous les mètres.

1. Quel est le coût du forage d'un puits de dix mètres de profondeur ?
2. Le budget alloué ne doit pas dépasser 700 €. Il écrit le script suivant.

```
1 profondeur = 1
2 prix = 100
3 while prix <= 700:
4     prix = prix + 20
5     profondeur = profondeur + 1
```

- a. Quelle est la valeur de la variable `profondeur` après l'exécution du script ?
- b. Quelle profondeur maximale pourra atteindre le jardinier avec son budget ?

Exercice 3

On lance deux dés à six faces parfaitement équilibrées et on additionne les deux résultats obtenus. La fonction suivante modélise les lancers et simule le nombre n de lancers qu'il a fallu pour obtenir la somme 12.

```
1 from random import randint
2 def somme12():
3     s = 0
4     n = 0
5     while s != 12:
6         de1 = randint(1, 6)
7         de2 = randint(1, 6)
8         s = de1 + de2
9         n = n + 1
10    return n
```

1. Ouvrir ou copier la fonction dans l'éditeur Python.
2. Tester cette fonction trois fois et indiquer le nombre de lancers obtenu.

Exercice 4

1. Écrire dans l'éditeur Python la fonction écrite ci-dessous.

```

1 def mystere(x, y):
2     z = 0
3     while x != 0:
4         if x%2 == 0:
5             x = x // 2
6             y = 2 * y
7         else:
8             x = x - 1
9             z = z + y
10    return z

```

2. Compléter le tableau à l'aide du script.

x	y	z (résultat)
2	5	
5	15	
2	-8	
100	0,01	
5	2,3	

3. Que fait cette fonction ?

Exercice 5

Un DAB (Distributeur Automatique de Billets) propose uniquement des billets de 10 € ou 20 €. Lors d'un retrait, le DAB distribue le moins de billets possible. On considère la fonction suivante :

```

1 def DAB(n):
2     if n%10 != 0:
3         return 'impossible'
4     else:
5         i = 0
6         while n >= 20:
7             n, i = n - 20, i + 1
8         billets_20 = i
9         billets_10 = n // 10
10    return billets_10, billets_20

```

1. À quoi sert l'instruction de la ligne 2 ?
2. Combien de billets de 10 € et de 20 € obtient-on pour un retrait de 330 € ?